



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS
INTEGRADO AO ENSINO MÉDIO

RAFAEL GOMES EMILIANO

KAIQUE AVANTE

PYETRO NUNES SAVIO

PEREIRA

TURMA 213

Documentação 3 Bimestre Sabor e Clic

SÃO PAULO 2026

PYETRO NUNES MARTINS

RAFAEL GOMES EMILIANO

KAIQUE AVANTE MOREIRA

SAVIO PEREIRA SOARES

SABOR E CLIC

Documentação do Projeto Integrador do Terceiro Bimestre

Documentação do Projeto Integrador do terceiro bimestre apresentada ao Curso Técnico em Desenvolvimento de Sistemas Integrado ao Ensino Médio do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo.

São Paulo

2026

SUMÁRIO

1 APRESENTAÇÃO DO PROJETO	4
2 OBJETIVOS E ESCOPO	4
2.1 Objetivo geral	4
2.2 Objetivos específicos	5
2.3 Limites desta etapa.....	5
3 CONCEPÇÃO DO SISTEMA E INTEGRAÇÃO ENTRE DISCIPLINAS	5
4 ATORES E RESPONSABILIDADES.....	6
5 FLUXO GERAL DO PRODUTO	7
6 RESERVA	7
6.1 Lógica de negócio	7
6.2 Contribuição de cada disciplina	8
7 SESSÃO	8
8 CARDÁPIO E CARRINHO	9
8.1 Lógica do carrinho	9
9 PEDIDOS.....	9
10 KDS	10
11 ADMINISTRAÇÃO DE BANCADAS E CATÁLOGO	11
12 ARQUITETURA DA APLICAÇÃO	11
13 LÓGICA JAVASCRIPT	12
14 ARQUITETURA FLASK E ROTAS ESSENCIAIS	13
14.1 Exemplo atual de rota Flask.....	14
15 BANCO DE DADOS E DER	14
15.1 Dicionário de Dados	15
16 SEGURANÇA E NÍVEIS DE ACESSO	24
17 IDENTIDADE VISUAL E PROTOTIPAÇÃO	26
17.1 Telas do perfil Cliente.....	27
17.2 Telas do perfil Cozinheiro	31
17.3 Telas do perfil Administrador.....	33
18 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS.....	38
19 TECNOLOGIAS UTILIZADAS.....	38
20 O QUE CONCLUÍMOS E PRÓXIMAS ETAPAS	39
20.1 O que concluímos	39
20.2 O que iniciamos	39
20.3 Próximas etapas	39
21 ATENDIMENTO AOS REQUISITOS DO TERCEIRO BIMESTRE	39
22 ESTADO ATUAL DA IMPLEMENTAÇÃO E INTEGRAÇÃO	40
22.1 Front-end e JavaScript	40
22.2 Back-end Flask em memória	40
22.3 Integração e próxima etapa	41
REFERÊNCIAS.....	41

1 APRESENTAÇÃO DO PROJETO

Nós desenvolvemos o Sabor e Clic como uma proposta de sistema para organizar a operação de uma cozinha compartilhada com Bancadas reserváveis e atendimento gastronômico integrado. Nosso ponto de partida foi um problema recorrente: quando Reservas, disponibilidade de espaços, Cardápio e Pedidos são controlados por planilhas ou anotações isoladas, surgem conflitos de horário, falta de rastreabilidade e dificuldade para acompanhar a produção. Por isso, concebemos uma experiência única que liga o Cliente à cozinha e à administração.

Nossa documentação acompanha o fluxo do produto: Reserva → Sessão → Cardápio e Pedidos → KDS. Escolhemos esse encadeamento porque cada Pedido precisa existir dentro de um contexto de atendimento. A Reserva define quem utilizará qual Bancada e em qual período; a Sessão representa o uso efetivo dessa Reserva; o Cardápio permite selecionar itens; o Pedido registra a solicitação; e o KDS oferece à cozinha uma visão organizada da produção. Ao longo do terceiro bimestre, reunimos contribuições de Artes Visuais e Design, JavaScript, Python, Banco de Dados e Segurança. Nosso protótipo estático demonstra as principais telas e interações. A modelagem de dados descreve as entidades e os relacionamentos necessários para sustentar a aplicação. Também implementamos uma API

Flask executável com dados simulados em memória e rotas JSON para o fluxo principal.

2 OBJETIVOS E ESCOPO

2.1 Objetivo geral

Nosso objetivo é construir um sistema web que permita administrar Bancadas gastronômicas, criar Reservas, iniciar Sessões, consultar o Cardápio, enviar Pedidos e acompanhar seu preparo no KDS. Pretendemos centralizar dados que normalmente estariam dispersos e oferecer uma visão adequada para Cliente, Cozinheiro e Administrador.

2.2 Objetivos específicos

- Representar o fluxo completo do atendimento de forma clara e navegável.
- Modelar as entidades, atributos e relacionamentos em um DER e em um dicionário de dados.
- Aplicar JavaScript moderno para manipular o DOM, manter o carrinho e calcular valores em memória.
- Definir uma API Flask organizada por responsabilidades, com comunicação por JSON.
- Planejar autenticação, validação e RBAC para proteger ações e dados.
- Criar identidade visual coerente e interfaces para Cardápio Digital e KDS.

2.3 Limites desta etapa

Nesta etapa concluímos o protótipo estático, a documentação conceitual de dados e uma API Flask executável com dados mantidos em memória. O front-end ainda não utiliza Fetch e não está integrado à API. Banco físico, autenticação real, cookies de login, RBAC no servidor, POO completa em JavaScript e persistência transacional permanecem para o quarto bimestre.

3 CONCEPÇÃO DO SISTEMA E INTEGRAÇÃO ENTRE DISCIPLINAS

Nós organizamos o projeto como um sistema interdisciplinar. Design define como o usuário compreende estados e ações; JavaScript transforma a interface em uma experiência dinâmica; Flask expõe contratos JSON e aplica as validações desta etapa; Banco de Dados descreve a persistência futura por meio dos modelos conceituais; e Segurança define identidade, permissão e regras de entrada. A qualidade do fluxo depende da concordância entre essas partes.

Quadro 1 – Contribuições das disciplinas e evolução arquitetural

Disciplina	Contribuição realizada	Evolução arquitetural
------------	------------------------	-----------------------

Disciplina	Contribuição realizada	Evolução arquitetural
ARTES VISUAIS DIGITAIS	Paleta, tipografia, componentes, telas por perfil, Cardápio e KDS.	Fidelidade ao protótipo, acessibilidade e testes de usabilidade.
DESENVOLVIMENTO WEB 1	JavaScript modularizado por responsabilidade, DOM, carrinho, validações, cálculos em memória e perfis simulados.	Classes ES6+, Fetch/JSON, async/await e integração real com Flask no quarto bimestre.
LABORATÓRIO DE PROGRAMAÇÃO 1	Factory Flask, Blueprints, Routes, Controllers, Models em memória e endpoints JSON para Reservas, Sessões, Cardápio, Pedidos e KDS.	Persistência relacional, autenticação/RBAC, integração Fetch e services adicionais quando necessários.

Disciplina	Contribuição realizada	Evolução arquitetural
BANCO DE DADOS	DER, MER, EER e dicionário de dados.	Modelo físico, scripts SQL, consultas e triggers.
SEGURANÇA E DESENVOLVIMENTO DE SOFTWARE	Definição de perfis e separação visual.	Autenticação, cookies de login, RBAC no servidor e sanitização.

Fonte: elaboração da equipe (2026).

4 ATORES E RESPONSABILIDADES

Quadro 2 – Atores e responsabilidades

Ator	Objetivo no sistema	Principais ações
Cliente	Utilizar uma Bancada e consumir durante uma Sessão.	Consultar Bancadas, reservar, abrir Sessão, montar carrinho, enviar e acompanhar Pedidos.
Cozinheiro	Organizar a produção.	Consultar fila, iniciar preparo, marcar Pedido pronto e entregue, acompanhar tempos.
Administrador	Configurar e supervisionar a operação.	Gerenciar Bancadas, Reservas, Pratos, Ingredientes, usuários e relatórios.

Fonte: elaboração da equipe (2026).

Nós representamos os três atores por menus distintos. Essa separação melhora a compreensão do protótipo, mas sabemos que visibilidade de menu não equivale a autorização. Na aplicação integrada, cada requisição protegida precisará ser validada pelo Flask de acordo com o usuário autenticado e seu papel.

5 FLUXO GERAL DO PRODUTO

O fluxo começa quando o Cliente consulta uma data e um intervalo. O sistema compara capacidade e disponibilidade das Bancadas. Com uma opção válida, cria a Reserva e preserva o período e o valor acordado. No início do atendimento, a Reserva gera uma Sessão única. É a Sessão ativa que autoriza o acesso ao Cardápio e a criação de Pedidos. A cozinha recebe o Pedido no KDS, altera seu estado e devolve esse progresso ao Cliente. No encerramento, a Sessão deixa de aceitar novos Pedidos e conserva o histórico.

Quadro 3 – Etapas do fluxo geral do produto

Etapa	Estado de entrada	Processamento	Saída
Reserva	Bancada disponível.	Validamos período, capacidade e conflito.	Reserva confirmada.
Sessão	Reserva válida.	Criamos contexto único e ativamos o atendimento.	Sessão ativa.
Cardápio	Sessão ativa.	Exibimos Pratos disponíveis e mantemos carrinho.	Itens selecionados.
Pedido	Carrinho válido.	Revalidamos itens, preços e quantidades.	Pedido recebido.
KDS	Pedido recebido.	Organizamos filas e controlamos transições.	Pedido em preparo, pronto e entregue.

Fonte: elaboração da equipe (2026).

Esse fluxo também orienta a estrutura de dados. O Pedido não precisa repetir Cliente ou Bancada, porque sua Sessão conduz à Reserva e a Reserva identifica ambos. Essa decisão reduz a redundância e mantém uma única fonte para o contexto do atendimento.

6 RESERVA

6.1 Lógica de negócio

Nós tratamos a Reserva como o compromisso de uso de uma Bancada durante um intervalo. Para criar uma Reserva, precisamos identificar o Cliente, a Bancada, o início, o fim, o número de pessoas e o valor. A Bancada deve estar ativa, comportar o grupo e não possuir outra Reserva incompatível. O fim precisa ser posterior ao início.

Há sobreposição quando o início solicitado ocorre antes do fim existente e o fim solicitado ocorre depois do início existente. A API em memória já executa essa verificação antes de criar a Reserva. No quarto bimestre, a mesma regra deverá ocorrer dentro da transação do banco para evitar conflitos entre solicitações simultâneas.

6.2 Contribuição de cada disciplina

Em Design, criamos cartões de Bancada, etiquetas de disponibilidade, filtros e formulário de Reserva. Em JavaScript, utilizamos o array benches e templates para renderizar capacidade, recursos, preço e estado. Em Banco de Dados, relacionamos CLIENTE e BANCADA à RESERVA. Em Flask, implementamos consulta de Bancadas, criação e

consulta de Reservas com validações em memória. Autorização por proprietário e perfil permanece planejada para o quarto bimestre.

Trecho 1 – Estrutura real utilizada para representar Bancadas no protótipo

```
const benches = [  
  {id:1,   nome:"Bancada   Toscana",   cap:4,   preco:85,   status:"Disponível",  
    recursos:["Cooktop","Pia","Tomadas"]}  
];
```

Fonte: código do protótipo Sabor e Clic (2026).

No navegador, os dados permanecem em memória e os botões simulam confirmação. Separadamente, a API Flask já recebe JSON, válida capacidade, datas e conflitos, e devolve

respostas JSON com códigos 201, 400, 404 ou 409. A ligação dessas duas partes por Fetch fica para o quarto bimestre.

7 SESSÃO

Nós adotamos Sessão como entidade central do atendimento. Uma Reserva descreve o período contratado; a Sessão registra o uso efetivo e reúne os Pedidos daquele período. O model em memória garante uma Sessão por Reserva e registra identificador, código de acesso, estado e datas de criação, início e encerramento.

Nesta implementação, a Sessão é criada como ativa e pode ser encerrada por endpoint próprio. Somente uma Sessão ativa aceita Pedidos, e o Flask verifica esse estado em cada criação. Autorização por usuário e validade temporal serão acrescentadas com a autenticação real.

Na tela que desenvolvemos, mostramos tempo restante, quantidade de Pedidos, consumo, pessoas, Pedidos vinculados e ações de navegação. Em JavaScript, essa tela é uma função do objeto views. No back-end, SESSAO referencia uma RESERVA e origina vários PEDIDOS por id_sessao. No banco futuro, essa relação será protegida por chaves e unicidade.

8 CARDÁPIO E CARRINHO

Nosso Cardápio Digital apresenta foto, nome, descrição, categoria, preço e disponibilidade. Criamos oito produtos demonstrativos e organizamos a experiência em Entradas, Pratos, Bebidas e Sobremesas. A interface desabilita itens indisponíveis e a API repete essa validação ao criar um Pedido, usando o catálogo mantido em memória.

8.1 Lógica do carrinho

Trecho 2 – Adição de item e cálculo do total no protótipo

```
function addCart(id){  
  
    const p=products.find(x=>x.id===id); if(p.status!=="Disponível") return; const  
ex=state.cart.find(x=>x.id===id); if(ex) ex.qtd++;  
  
    else state.cart.push({id:p.id,nome:p.nome,preco:p.preco,qtd:1});
```

```

}

function cartTotal(){

return state.cart.reduce((s,i)=>s+i.preco*i.qtd,0);

}

```

Fonte: código do protótipo Sabor e Clic (2026).

Primeiro localizamos o produto pelo identificador. Se ele estiver indisponível, encerramos a função. Depois procuramos o mesmo item no carrinho: quando existe, incrementamos a quantidade; quando não existe, adicionamos um novo objeto. O total é calculado com reduce, acumulando preço multiplicado pela quantidade. Essa lógica satisfaz o objetivo do terceiro bimestre de manipular arrays, objetos e cálculos em memória.

A API recebe apenas identificadores e quantidades, consulta os preços atuais do catálogo em memória, valida cada Prato e calcula os subtotais e o total. O front-end ainda não envia esses dados por Fetch; essa integração permanece para o quarto bimestre.

9 PEDIDOS

O Pedido agrupa itens solicitados durante uma Sessão. Definimos que ele deve possuir ao menos um ITEM_PEDIDO, estado, valor total, tempo estimado e data de criação. ITEM_PEDIDO preserva quantidade, preço unitário praticado e subtotal. Mesmo que o preço do catálogo mude posteriormente, o histórico continua representando a compra original.

Ao confirmar um Pedido nesta etapa, o servidor verifica a existência e o estado ativo da Sessão, a lista não vazia, as quantidades positivas e a disponibilidade dos Pratos. Em seguida, busca os preços, calcula os subtotais e devolve JSON. Autenticação, propriedade da Sessão e transação em banco serão implementadas no quarto bimestre.

Quadro 4 – Estados dos Pedidos e próximas ações

Estado	Significado	Próxima ação esperada
AGUARDANDO ou RECEBIDO	Pedido aceito e visível para a cozinha.	Iniciar preparo ou cancelar conforme regra.

Estado	Significado	Próxima ação esperada
EM PREPARO	Produção iniciada.	Marcar como pronto.
PRONTO	Pedido finalizado pela cozinha.	Marcar como entregue.
ENTREGUE	Ciclo operacional concluído.	Somente consulta histórica.
CANCELADO	Pedido interrompido.	Somente consulta ou procedimento administrativo.

Fonte: elaboração da equipe (2026).

10 KDS

O Kitchen Display System é o painel do Cozinheiro. Nós o organizamos em colunas para tornar o estado imediatamente visível: Recebidos, Em preparo e Prontos. Cada ticket apresenta número, Bancada, tempo e itens. Cores e etiquetas reforçam prioridade e atraso. Essa estrutura reduz a necessidade de abrir cada Pedido separadamente.

No protótipo, os tickets continuam sendo exemplos visuais. A API Flask já oferece consultas JSON para a fila do KDS e inclui o contexto de Sessão, Reserva e Bancada de cada Pedido. A interface ainda não consome essa fila. No quarto bimestre, o KDS usará Fetch e as mudanças de estado serão persistidas e validadas pelo servidor.

Design contribui com hierarquia, cores e leitura rápida. O JavaScript controla a renderização demonstrativa sem F5. O Flask monta a fila em memória a partir dos Pedidos e de seus vínculos. O Banco registrará estados e horários no quarto bimestre, e a Segurança limitará as ações a Cozinheiro e Administrador quando a autenticação real for implementada.

11 ADMINISTRAÇÃO DE BANCADAS E CATÁLOGO

O Administrador configura os recursos utilizados nos fluxos anteriores. Desenvolvemos telas para Bancadas, Reservas, Produtos, Ingredientes, substituições, usuários e relatórios. As operações exibidas ainda são demonstrativas, mas definem a experiência e os dados necessários para os futuros CRUDs.

A exclusão de Prato deve ser lógica por meio do atributo ativo. Dessa forma, um item deixa de aparecer para novas compras sem destruir ITEM_PEDIDO histórico. A ficha técnica relaciona Prato e Ingrediente com a quantidade necessária. A substituição é direcional e possui prioridade e fator de conversão. A futura disponibilidade poderá somar o consumo de todos os itens e escolher substitutos autorizados quando o ingrediente original estiver insuficiente.

12 ARQUITETURA DA APLICAÇÃO

Nós definimos uma arquitetura em camadas. O navegador contém HTML, CSS e JavaScript; a API utiliza HTTP e JSON; as Routes Flask interpretam as requisições; os Controllers validam e executam os casos de uso; e os Models mantêm os dados demonstrativos em memória. O banco relacional será conectado apenas pelo back-end no quarto bimestre.

Quadro 5 – Camadas da aplicação

Camada	Responsabilidade	Exemplo
View	Apresentar conteúdo e capturar eventos.	Cardápio, formulário e KDS.
JavaScript de página	Gerenciar o estado visual; futuramente chamar a API.	Carrinho, filtros e feedback.
Routes Flask	Interpretar HTTP, JSON e códigos de resposta.	POST /api/reservas.
Controllers	Validar e executar casos de uso.	Verificar conflito e criar Sessão.
Models em memória	Representar e consultar dados demonstrativos.	Reserva, Sessão, Pedido e Prato.
Banco relacional futuro	Garantir persistência e integridade no quarto bimestre.	PK, FK, unicidade e transações.

Fonte: elaboração da equipe (2026).

O repositório separa o front-end em módulos JavaScript e o back-end em Routes, Controllers e Models. No navegador, dados simulados sustentam as telas e os fluxos. No servidor, Blueprints expõem endpoints reais e os Controllers aplicam regras sobre Models em

memória. As duas partes ainda funcionam separadamente; a integração por Fetch e a persistência ficam para a próxima etapa.

13 LÓGICA JAVASCRIPT

O JavaScript atual foi dividido por responsabilidade para evitar um arquivo monolítico e tornar explícitos os conteúdos do terceiro bimestre. `data.js` concentra arrays, objetos literais e estado compartilhado; `site.js` reúne navegação, `localStorage` e helpers; `components.js`

contém os geradores visuais; `catalog.js` manipula o DOM do Cardápio; `cart.js` mantém o carrinho e calcula valores; `validation.js` valida Reservas; `login.js` trata o login demonstrativo; `views.js` registra somente as telas; e `a11y.js` concentra comportamentos de acessibilidade.

Trecho 3 – Renderização dinâmica do Cardápio utilizando `forEach` e manipulação do DOM

```
function renderCatalog(category = "Todos") {  
  
  const grid = document.querySelector(".grid.cols-4"); grid.innerHTML = "";  
  
  products.forEach(product => {  
  
    if (category !== "Todos" && product.cat !== category) return; const wrapper =  
document.createElement("div"); wrapper.innerHTML = productCard(product).trim();  
grid.appendChild(wrapper.firstElementChild);  
  
    });  
  
  }
```

Fonte: código do protótipo Sabor e Clic (2026).

Utilizamos arrays, objetos literais, `const`, arrow functions, template literals, `find`, `filter`, `reduce`, `forEach`, `for...of` e `for...in`. O Cardápio é atualizado dinamicamente pelo DOM; o carrinho mantém itens e calcula subtotais e total em memória; os formulários possuem validações simples; e o login demonstrativo seleciona o perfil e armazena a sessão localmente. Esses elementos cobrem os conteúdos de JavaScript previstos para o terceiro bimestre.

No quarto bimestre, a modularização será ampliada com classes ES6+, encapsulamento e módulos voltados à comunicação com a API. As chamadas Fetch serão assíncronas, utilizarão JSON e serão tratadas com `async/await` e `try/catch`. O estado que hoje é simulado no navegador passará a refletir as respostas reais do Flask.

14 ARQUITETURA FLASK E ROTAS ESSENCIAIS

O back-end do terceiro bimestre é uma aplicação Flask executável. A factory usa `Flask(__name__)`, carrega a configuração e registra Blueprints. As Routes usam `request.get_json()` e `jsonify`; os Controllers validam os casos de uso; e os Models mantêm usuários, Bancadas, Reservas, Sessões, Pratos e Pedidos em memória. Não há banco físico, autenticação real ou integração Fetch nesta etapa.

Quadro 6 – Rotas essenciais da API Flask

Método e rota	Atores	Lógica essencial
GET /api	Todos	Confirma que a API está em execução.
POST/GET /api/cadastro	Cliente	Cria e consulta cadastro demonstrativo.
POST /api/login	Todos	Identifica perfil sem autenticação real.
GET /api/bancadas	Cliente/Admin	Lista Bancadas em memória.
POST/GET /api/reservas[/<id>]	Cliente/Admin	Cria e consulta Reservas em memória.
POST/GET /api/sessoes[/<id>]	Cliente/Admin	Cria e consulta Sessões em memória.
POST /api/sessoes/<id>/encerrar	Cliente/Admin	Encerra Sessão e bloqueia novos Pedidos.
GET /api/cardapio	Cliente	Retorna o Cardápio em memória.
POST/GET /api/pedido[/cadastro/<id>]	Cliente	Cria por id_sessao e consulta Pedidos.
GET /api/kds[/<id>]	Cozinheiro/Admin	Lista a fila ou retorna um Pedido com Bancada.

Fonte: elaboração da equipe (2026).

14.1 Exemplo atual de rota Flask

Trecho 4 – Rota Flask executável para criação de Reserva

```
@reserva_bp.post("/reservas") def reservas():
```

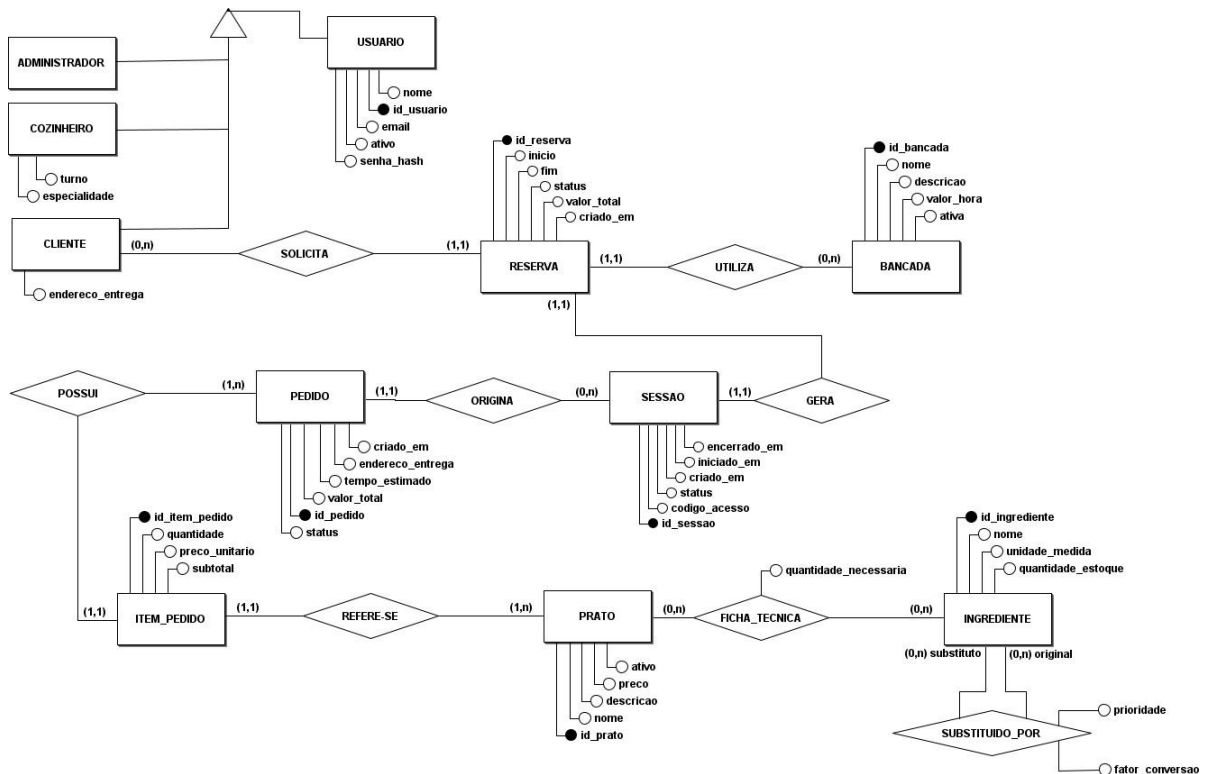
```
    dados = request.get_json(silent=True) or {}
    resposta, status = criar_reserva(dados)
    return jsonify(resposta), status
```

Fonte: código da API Sabor e Clic (2026).

15 BANCO DE DADOS E DER

Nós modelamos USUARIO e suas especializações CLIENTE, COZINHEIRO e ADMINISTRADOR; BANCADA e RESERVA; SESSAO, PEDIDO e ITEM_PEDIDO; PRATO e INGREDIENTE; além de FICHA_TECNICA e SUBSTITUIDO_POR. O dicionário descreve atributos, tipos sugeridos, domínios e regras de consistência.

Figura 1 – Diagrama Entidade-Relacionamento do Sabor e Clic



Fonte: elaboração da equipe (2026).

A cadeia $RESERVA \leftrightarrow SESSAO \rightarrow PEDIDO \rightarrow ITEM_PEDIDO$ é o núcleo do modelo. **CLIENTE** solicita **RESERVA** e **BANCADA** é utilizada pela **RESERVA**. Cada item referencia um **PRATO**. **FICHA_TECNICA** resolve a relação muitos para muitos entre Prato e Ingrediente, enquanto **SUBSTITUIDO_POR** utiliza duas referências para Ingrediente.

15.1 Dicionário de Dados

Para complementar a modelagem apresentada na seção de Banco de Dados, incorporamos integralmente o Dicionário de Dados do Sabor e Clic.

O material a seguir descreve as entidades, os atributos, os tipos sugeridos, os relacionamentos e as regras de consistência do sistema.

A inclusão integral preserva o conteúdo técnico elaborado pela equipe e mantém o Dicionário como referência para a implementação do modelo físico e das regras de negócio nas próximas etapas do projeto.

15.1.1 INTRODUÇÃO

Este documento apresenta o dicionário de dados do sistema Sabor & Clic, destinado à gestão de bancadas gastronômicas, reservas, sessões de atendimento, pedidos, pratos e ingredientes. Sua finalidade é descrever o significado dos atributos e os vínculos entre as entidades, apoiando a leitura do modelo conceitual e a futura implementação do banco de dados.

A elaboração atende à etapa 1 de Banco de Dados prevista no item 5.4 do Projeto Integrador, versão de 31 de agosto de 2026, que solicita o DER no brModelo e o dicionário de dados em formato .docx. A fonte dos nomes de entidades, atributos e relacionamentos é o arquivo der_saboreclic_final.png.

15.1.2 Convenções de leitura

Os atributos apresentados nas seções 2 a 7 correspondem ao DER. Tipos SQL, tamanhos, obrigatoriedade, estados e validações descritos são propostas de implementação, pois o modelo conceitual não define esses detalhes. Os identificadores preenchidos no diagrama são apresentados como chaves primárias sugeridas (PK). Os vínculos que poderão gerar chaves estrangeiras (FK) são tratados na seção 9, sem acrescentá-los à lista de atributos conceituais.

INT representa números inteiros; VARCHAR(n), texto de até n caracteres; TEXT, texto extenso; DECIMAL(p,s), valores exatos com p algarismos e s casas decimais; DATETIME, data e hora; BOOLEAN, verdadeiro ou falso. Valores monetários são expressos em reais. Datas e horários devem adotar o mesmo fuso na aplicação.

Salvo indicação de que um campo pode ficar vazio, propõe-se seu preenchimento obrigatório. Os identificadores não devem ser reutilizados. Totais e subtotais devem ser calculados e conferidos pelo sistema. O histórico de pedidos é obtido pelos relacionamentos, sem atributo adicional no cliente.

15.1.3 USUÁRIOS E PERFIS

USUARIO:

Reúne os dados comuns dos usuários que acessam o sistema.

Atributo	Tipo sugerido	Descrição e domínio
id_usuario	INT	Identificador único, obrigatório e positivo (PK sugerida).
nome	VARCHAR(150)	Nome completo do usuário.
email	VARCHAR(254)	Endereço de acesso válido e único.
ativo	BOOLEAN	Indica se o acesso está habilitado.
senha_hash	VARCHAR(255)	Representação da senha protegida por hash; nunca armazenar a senha em texto puro.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

Especializações de USUARIO

ADMINISTRADOR, COZINHEIRO e CLIENTE herdam o identificador e os atributos de USUARIO. ADMINISTRADOR não possui atributos exclusivos no DER. Sua função é administrar o catálogo. A exclusividade ou sobreposição dos perfis deve ser definida na implementação, pois não está explicitada no diagrama.

Atributo	Tipo sugerido	Descrição e domínio
COZINHEIRO.turno	VARCHAR(30)	Turno de trabalho; por exemplo, manhã, tarde ou noite.
COZINHEIRO.especialidade	VARCHAR(100)	Área de atuação culinária.
CLIENTE.endereco_entrega	VARCHAR(255)	Endereço padrão do cliente. Pode ficar vazio em atendimento exclusivo na bancada.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

15.1.4 BANCADAS E RESERVAS

BANCADA

Representa o espaço gastronômico disponibilizado para locação.

Atributo	Tipo sugerido	Descrição e domínio
id_bancada	INT	Identificador único, obrigatório e positivo (PK sugerida).
nome	VARCHAR(100)	Nome de identificação da bancada.
descricao	TEXT	Características e equipamentos disponíveis.
valor_hora	DECIMAL(10,2)	Preço por hora, maior ou igual a zero.
ativa	BOOLEAN	Indica se a bancada pode receber novas reservas.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

RESERVA

Registra o período contratado por um cliente para uma bancada.

Atributo	Tipo sugerido	Descrição e domínio
id_reserva	INT	Identificador único, obrigatório e positivo (PK sugerida).
inicio	DATETIME	Data e hora previstas para início.
fim	DATETIME	Data e hora previstas para término; posterior ao início.
status	VARCHAR(20)	Proposta: CONFIRMADA, EM_USO, ENCERRADA ou CANCELADA.
valor_total	DECIMAL(10,2)	Valor contratado para a locação, não negativo; preserva o histórico de preço.
criado_em	DATETIME	Data e hora de criação do registro.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

15.1.5 SESSÕES E PEDIDOS

SESSAO

Contextualiza os pedidos dentro de uma reserva. Cada reserva gera uma sessão.

Atributo	Tipo sugerido	Descrição e domínio
id_sessao	INT	Identificador único, obrigatório e positivo (PK sugerida).
codigo_acesso	VARCHAR(128)	Código único de acesso à sessão; deve perder validade no encerramento.
status	VARCHAR(20)	Proposta: CRIADA, ATIVA ou ENCERRADA.
criado_em	DATETIME	Data e hora de criação da sessão.

Atributo	Tipo sugerido	Descrição e domínio
iniciado_em	DATETIME	Início efetivo; pode ficar vazio antes da ativação.
encerrado_em	DATETIME	Encerramento efetivo; pode ficar vazio enquanto não encerrada.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

15.1.6 PEDIDO

Agrupa os itens solicitados em uma sessão e acompanha seu preparo.

Atributo	Tipo sugerido	Descrição e domínio
id_pedido	INT	Identificador único, obrigatório e positivo (PK sugerida).
status	VARCHAR(20)	Proposta: AGUARDANDO, EM_PREPARO, PRONTO, ENTREGUE ou CANCELADO.
valor_total	DECIMAL(10,2)	Soma dos subtotais dos itens; valor não negativo.
tempo_estimado	INT	Estimativa de preparo em minutos; não negativa. Pode ficar vazia até ser definida.
endereco_entrega	VARCHAR(255)	Endereço usado neste pedido. Pode ficar vazio para entrega na bancada.
criado_em	DATETIME	Data e hora de criação do pedido.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

15.1.7 ITENS E PRATOS

ITEM_PEDIDO

Registra um prato solicitado e seu preço no momento da compra.

Atributo	Tipo sugerido	Descrição e domínio
----------	---------------	---------------------

Atributo	Tipo sugerido	Descrição e domínio
id_item_pedido	INT	Identificador único, obrigatório e positivo (PK sugerida).
quantidade	INT	Número de unidades solicitadas; maior que zero.
preco_unitario	DECIMAL(10,2)	Preço unitário aplicado ao pedido, não negativo; não acompanha futuras alterações no catálogo.
subtotal	DECIMAL(10,2)	Quantidade multiplicada pelo preço unitário; deve permanecer consistente com esses campos.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

PRATO

Representa um produto do cardápio oferecido ao cliente.

Atributo	Tipo sugerido	Descrição e domínio
id_prato	INT	Identificador único, obrigatório e positivo (PK sugerida).
nome	VARCHAR(100)	Nome de exibição no cardápio.
descricao	TEXT	Descrição do prato e de suas características.
preco	DECIMAL(10,2)	Preço atual de venda; maior ou igual a zero.
ativo	BOOLEAN	Indica se o prato está habilitado no catálogo.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

A inativação de um prato deve preservar os itens de pedidos anteriores. O preço registrado em ITEM_PEDIDO representa a venda realizada, enquanto o preço de PRATO representa o valor atual do catálogo.

15.1.8 INGREDIENTES E FICHA TÉCNICA

INGREDIENTE

Representa um insumo controlado em estoque e utilizado nas receitas.

Atributo	Tipo sugerido	Descrição e domínio
id_ingrediente	INT	Identificador único, obrigatório e positivo (PK sugerida).
nome	VARCHAR(100)	Nome do ingrediente.
unidade_medida	VARCHAR(10)	Unidade de controle, por exemplo: g, kg, ml, l ou un.
quantidade_estoque	DECIMAL(12,3)	Saldo disponível na unidade cadastrada; não negativo.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

FICHA_TECNICA

Relacionamento entre PRATO e INGREDIENTE que registra o consumo de cada insumo para uma unidade do prato.

Atributo	Tipo sugerido	Descrição e domínio
quantidade_necessaria	DECIMAL(12,3)	Quantidade do ingrediente consumida por unidade do prato, na unidade de controle do insumo; maior que zero.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

Um prato pode relacionar-se a vários ingredientes e um ingrediente pode integrar vários pratos. No modelo relacional, sugere-se uma tabela associativa com o par de identificadores de prato e ingrediente como chave composta, além da quantidade necessária.

Para uma solicitação de três unidades de um prato que utiliza 0,150 kg de um ingrediente por unidade, o consumo previsto será de 0,450 kg. O cálculo deve somar o consumo de todos os itens antes de confirmar a disponibilidade.

15.1.9 SUBSTITUIÇÕES DE INGREDIENTES

SUBSTITUIDO_POR

Autorrelacionamento de INGREDIENTE. Distingue o ingrediente original daquele autorizado como seu substituto.

Atributo	Tipo sugerido	Descrição e domínio
prioridade	INT	Ordem de preferência entre substitutos de um mesmo original. Proposta: 1 indica a maior prioridade.
fator_conversao	DECIMAL(12,6)	Quantidade do substituto para cada unidade do original; deve ser maior que zero.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

A quantidade necessária do substituto é calculada multiplicando a quantidade requerida do original pelo fator de conversão. O fator deve considerar as unidades de medida dos dois ingredientes. Exemplo: se a receita exige 2 unidades do pão original e o fator é 1,5, serão necessárias 3 unidades do pão substituto.

A autorização é direcional: permitir que A seja substituído por B não autoriza automaticamente substituir B por A. Recomenda-se impedir pares com o mesmo ingrediente nos dois papéis, evitar duplicação do par original–substituto e definir prioridades distintas para um mesmo original.

O DER permite vários substitutos por ingrediente e permite que um ingrediente substitua vários outros. A tabela relacional sugerida deve conter duas referências a INGREDIENTE, identificadas pelos papéis original e substituto, além dos atributos do relacionamento.

15.1.10 REGRAS DE CONSISTÊNCIA

As reservas da mesma bancada não devem ter períodos conflitantes. Novos pedidos devem pertencer a uma sessão ativa e conter ao menos um item. A baixa de estoque deve considerar as quantidades da ficha técnica e, quando aplicável, os substitutos autorizados. Essas validações são regras da aplicação e da futura implementação física, não novos atributos do DER.

15.1.11 RELACIONAMENTOS E MAPEAMENTO

O quadro apresenta a leitura dos vínculos do DER e uma proposta para representá-los no modelo relacional. Os nomes das FK são sugestões, pois não aparecem como atributos no diagrama conceitual.

Relacionamento	Entidades	Cardinalidade e mapeamento
SOLICITA	CLIENTE → RESERVA	Cliente: zero a várias reservas. Reserva: exatamente um cliente. Sugestão: RESERVA.id_cliente como FK obrigatória.
UTILIZA	BANCADA → RESERVA	Bancada: zero a várias reservas. Reserva: uma bancada. Sugestão: RESERVA.id_bancada como FK obrigatória.
GERA	RESERVA ↔ SESSAO	Uma sessão por reserva e uma reserva por sessão. Sugestão: SESSAO.id_reserva como FK obrigatória e única.
ORIGINA	SESSAO → PEDIDO	Sessão: zero a vários pedidos. Pedido: uma sessão. Sugestão: PEDIDO.id_sessao como FK obrigatória.
POSSUI	PEDIDO → ITEM_PEDIDO	Pedido: um ou vários itens. Item: um pedido. Sugestão: ITEM_PEDIDO.id_pedido como FK obrigatória.
REFERE-SE	PRATO → ITEM_PEDIDO	Cada item referencia um prato. O DER indica um ou vários itens por prato. Sugestão: ITEM_PEDIDO.id_prato como FK obrigatória.
FICHA_TECNICA	PRATO ↔ INGREDIENTE	Zero a vários em ambos os lados. Sugestão: tabela associativa com PK composta (id_prato, id_ingrediente).
SUBSTITUIDO_POR	INGREDIENTE ↔ INGREDIENTE	Zero a vários em ambos os papéis. Sugestão: PK composta (id_original, id_substituto), com duas FK para INGREDIENTE.

Fonte: elaboração própria com base no DER final do Sabor & Clic (2026).

15.1.12 OBSERVAÇÕES SOBRE O MODELO

O vínculo de um pedido com a reserva ocorre por meio de SESSAO. Portanto, o caminho PEDIDO → SESSAO → RESERVA identifica a bancada e o cliente associados ao pedido, sem exigir uma segunda referência direta à reserva.

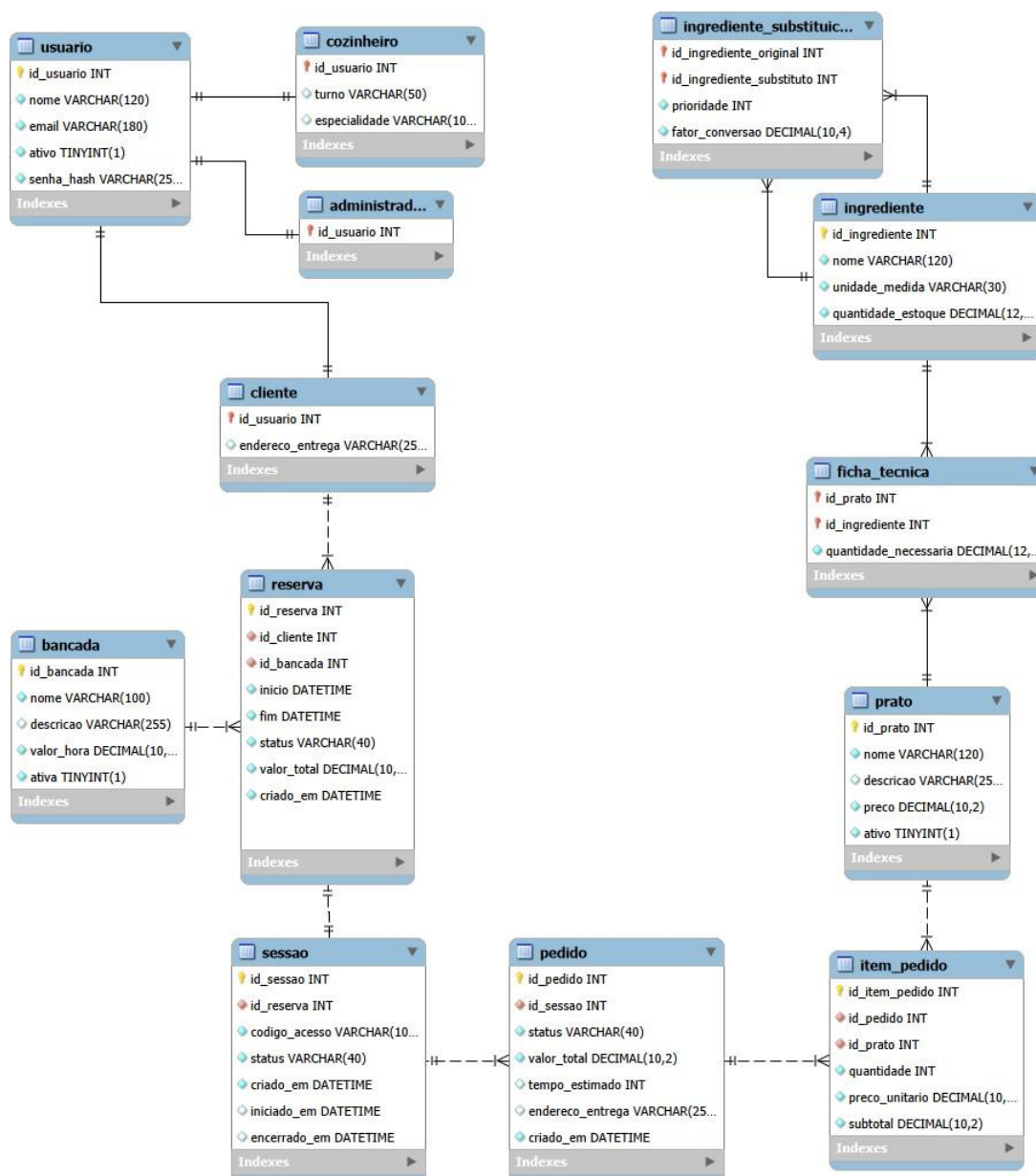
Na especialização, uma implementação possível é manter USUARIO e uma tabela para cada perfil. Cada tabela especializada utiliza id_usuario como chave primária e estrangeira para USUARIO. ADMINISTRADOR pode conter apenas esse identificador, mesmo sem atributos exclusivos.

O DER indica participação mínima de um item de pedido para PRATO no relacionamento REFERE-SE. Isso impediria conceitualmente a existência de pratos ainda não vendidos. Recomenda-se validar com o professor a alteração para zero a vários itens por prato, caso o cadastro de produtos preceda as vendas. O dicionário registra a cardinalidade apresentada, sem alterar o diagrama.

O relacionamento GERA é obrigatório nos dois lados. Uma FK única em SESSAO garante que cada sessão tenha uma reserva e impede duas sessões para a mesma reserva; a criação de uma sessão para toda reserva deve ser garantida também pelo fluxo transacional da aplicação.

O modelo identifica substituições permitidas, mas não registra qual substituto foi efetivamente consumido em cada pedido. Caso seja necessário auditar esse consumo, a etapa física deverá avaliar uma estrutura específica para movimentações de estoque. Essa estrutura não integra o DER documentado.

Figura 2 – Modelo de entidades e relacionamentos



Fonte: elaboração da equipe (2026).

Definimos regras de integridade: e-mail único, valores não negativos, fim posterior ao início, Sessão única por Reserva, quantidade positiva, Pedido com itens e chaves estrangeiras obrigatórias. No quarto bimestre, o modelo físico deverá traduzir essas regras para SQL, constraints, índices, inserções básicas, consultas e triggers de estoque.

16 SEGURANÇA E NÍVEIS DE ACESSO

Definimos RBAC com três papéis. Cliente acessa seus próprios dados; Cozinheiro opera Pedidos e KDS; Administrador gerencia cadastros e visão geral. No protótipo, essa

divisão aparece nos menus. Na aplicação final, ela será aplicada no servidor por decorators ou middleware, porque ocultar um botão não impede uma chamada HTTP direta.

Quadro 7 – Riscos e decisões de segurança

Risco	Decisão arquitetural
Senha exposta	Armazenar hash forte com salt; nunca senha em texto ou cifra reversível.
Preço manipulado	Recalcular total no Flask a partir do catálogo.
Reserva duplicada	Validar sobreposição dentro de transação.
SQL Injection	Utilizar ORM ou consultas parametrizadas.
XSS	Escapar dados e preferir <code>textContent</code> para conteúdo externo.
CSRF/cookie	Configurar <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> e proteção CSRF quando aplicável.
Acesso indevido	Verificar identidade, propriedade do recurso e papel em cada rota.

Fonte: elaboração da equipe (2026).

Removemos o utilitário que apenas concatenava uma chave ao texto, pois ele não implementava criptografia. Nenhuma senha é armazenada pelo back-end demonstrativo. No quarto bimestre, a autenticação real deverá usar hash unidirecional com salt, mensagens genéricas para falhas de login e logs sem informações sensíveis.

17 IDENTIDADE VISUAL E PROTOTIPAÇÃO

Nós construímos uma identidade inspirada em materiais e cores gastronômicas. Utilizamos grafite #2B241E para estrutura, fundo #F6F1E8, cobre #BB6B2F como destaque, verde #4B7A52 para sucesso, açafrão #B9832A para atenção e páprica #A6432E para erro. Archivo aparece em títulos e Hanken Grotesk nos elementos de interface.

Para documentar a identidade visual e o recurso de acessibilidade, a tabela a seguir apresenta a paleta padrão e sua aparência aproximada quando o modo de auto contraste é ativado. No protótipo, esse modo não substitui as variáveis de cor por uma segunda paleta fixa: o CSS aplica contraste de 1,18 ao conteúdo da página, intensificando a separação entre tons claros, escuros e cores de estado.

Tabela 1 – Paleta de cores padrão e auto contraste

Função visual	Paleta padrão	Auto contraste*	Aplicação principal
Grafite / estrutura	#2B241E	#1C140C	Menu lateral, estrutura e elementos escuros
Fundo	#F6F1E8	#FFFFFFB	Plano de fundo principal
Cobre / destaque	#BB6B2F	#C66721	Ações principais, destaques e navegação ativa
Verde / sucesso	#4B7A52	#42794A	Estados positivos, disponibilidade e confirmação
Açafrão / atenção	#B9832A	#C3841B	Avisos, espera e estados intermediários
Páprica / erro	#A6432E	#AD381F	Erros, cancelamentos e ações de risco

* Valores aproximados para documentação, resultantes do filtro CSS de contraste aplicado sobre a paleta padrão.
Fonte: elaboração da equipe com base na implementação do front-end Sabor e Clic (2026).

Criamos componentes consistentes: menu lateral, cartões, badges, botões, tabelas, filtros, indicadores, timeline e colunas do KDS. CSS Grid e Flexbox organizam o conteúdo, enquanto media queries adaptam a interface. As imagens das Bancadas e dos produtos aproximam o protótipo de uma experiência de uso real.

O protótipo navegável do Sabor e Clic no Figma reúne os wireframes e as telas usadas como referência para o front-end. O material do protótipo contempla 21 telas, distribuídas entre os três perfis do sistema, e demonstra tanto o fluxo do cliente quanto as rotinas de cozinha e administração.

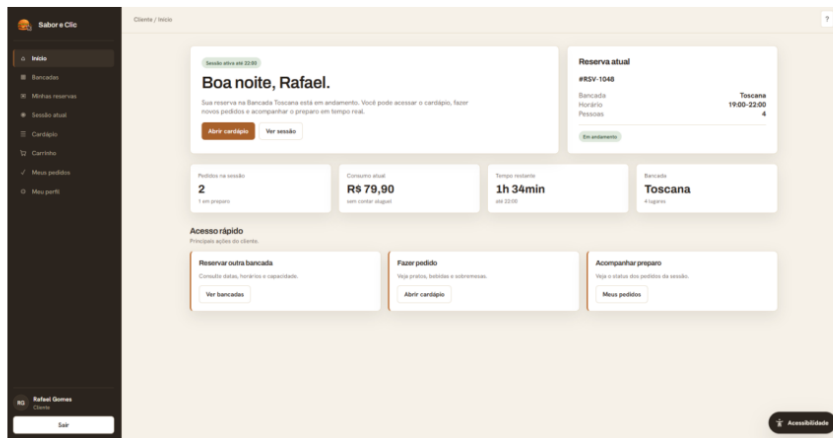
Cliente: Início, com resumo da reserva atual, indicadores da sessão e atalhos; Bancadas, com consulta de opções, filtros, capacidade, preço e disponibilidade; Minhas reservas, com histórico, situação e formulário para nova reserva; Sessão atual, com tempo restante, pedidos, consumo, pessoas e ações da sessão; Cardápio, com categorias e produtos disponíveis; Carrinho, com itens selecionados, observações, resumo e envio do pedido; Meus pedidos, com acompanhamento por linha do tempo; e Meu perfil, com dados pessoais e preferências.

Cozinheiro: Visão geral da cozinha, com indicadores e prioridades do momento; KDS, organizado em colunas de recebidos, em preparo e prontos; Pedidos, com listagem e filtros operacionais; Disponibilidade do cardápio, para visualizar e alterar o estado dos produtos; e Histórico, com registro dos pedidos já processados.

Administrador: Dashboard, com indicadores gerais da operação e alertas; Reservas, com filtros, consulta e criação; Bancadas, com cadastro e manutenção dos espaços; Produtos e pratos, com catálogo administrativo; Ingredientes, com estoque e movimentações; Substituições, com regras de ingredientes substitutos e prioridade; Usuários, com listagem e gerenciamento dos perfis; e Relatórios, com métricas consolidadas, como faturamento, pedidos, ticket médio, tempo médio e desempenho por bancada.

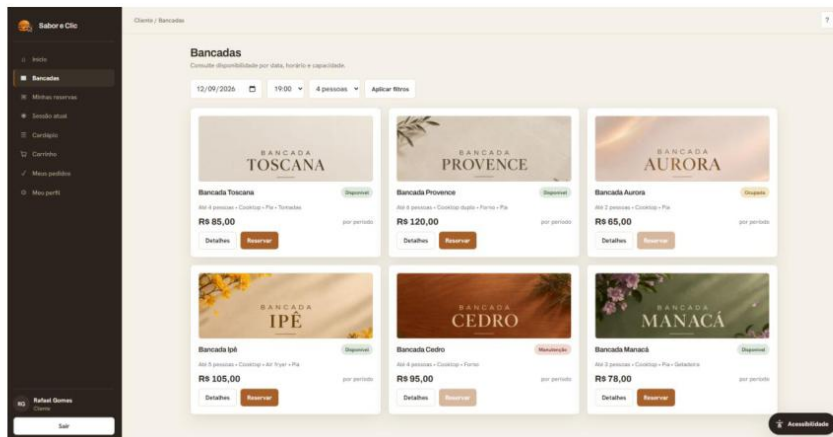
17.1 Telas do perfil Cliente

Figura 3 – Protótipo: Início (Cliente)



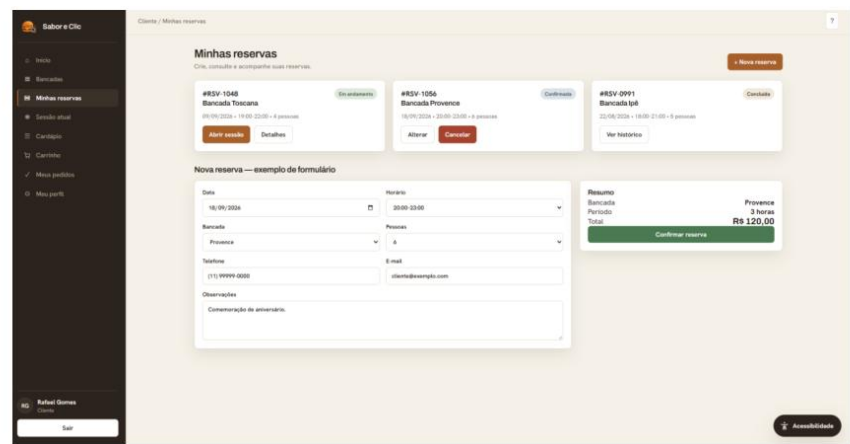
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 4 – Protótipo: Bancadas (Cliente)



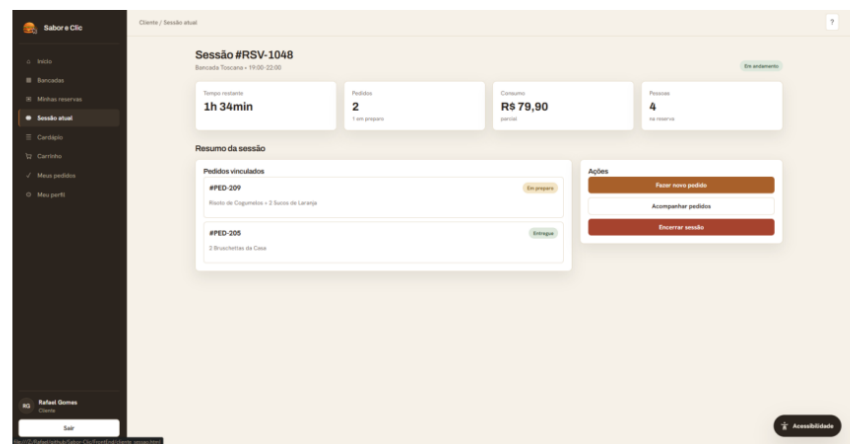
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 5 – Protótipo: Minhas reservas (Cliente)



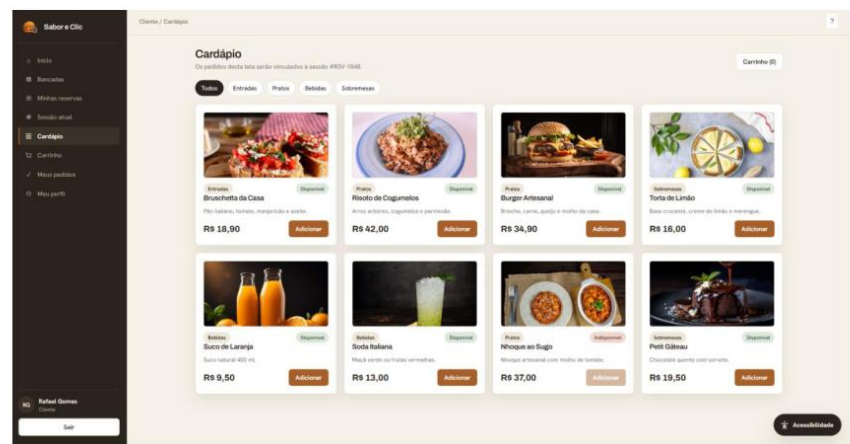
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 6 – Protótipo: Sessão atual (Cliente)



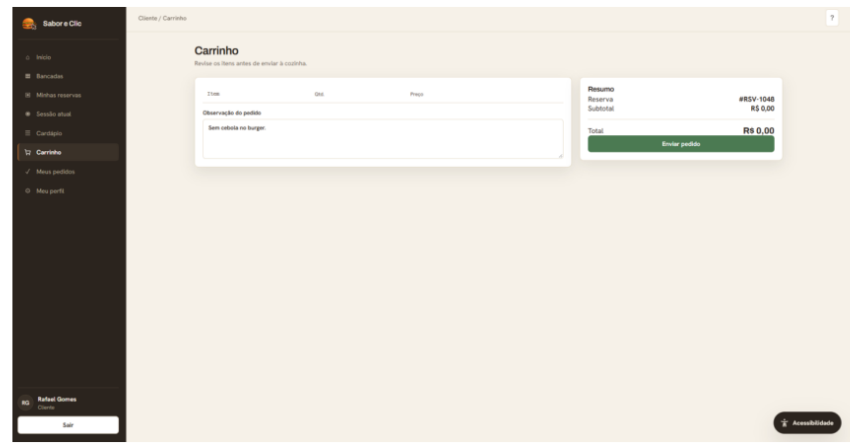
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 7 – Protótipo: Cardápio (Cliente)



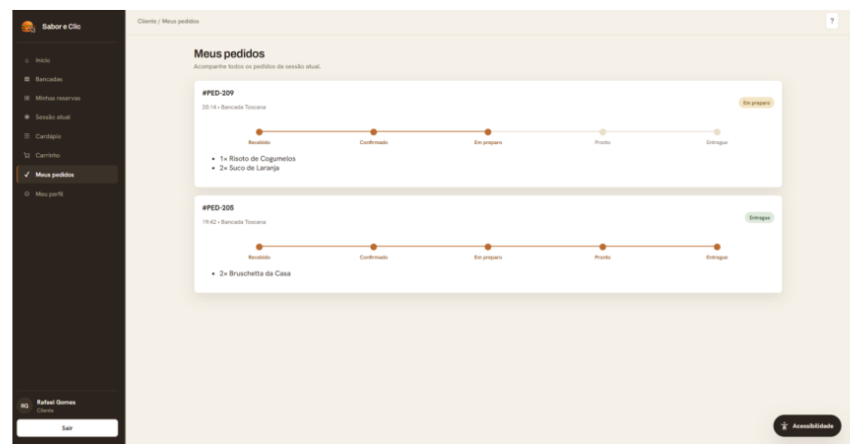
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 8 – Protótipo: Carrinho (Cliente)



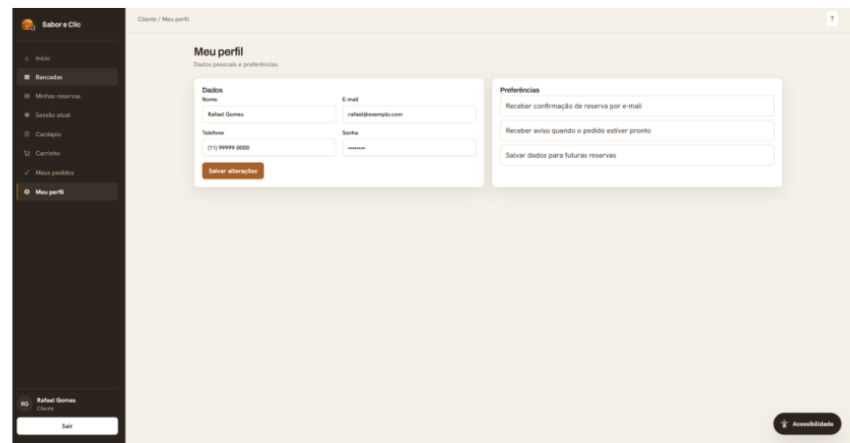
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 9 – Protótipo: Meus pedidos (Cliente)



Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

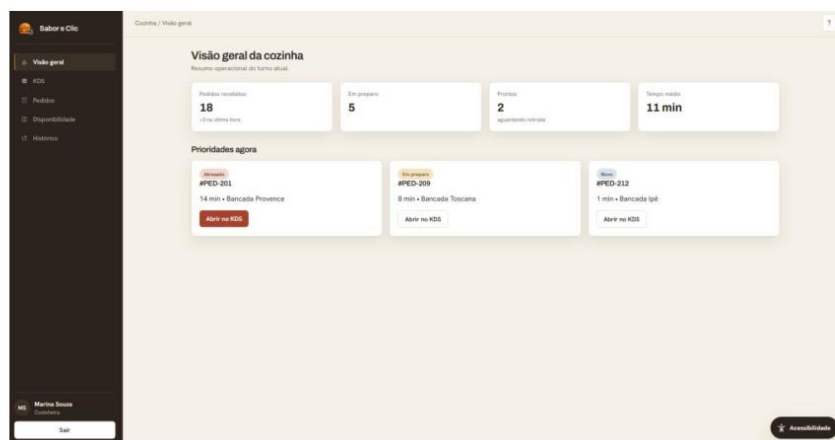
Figura 10 – Protótipo: Meu perfil (Cliente)



Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

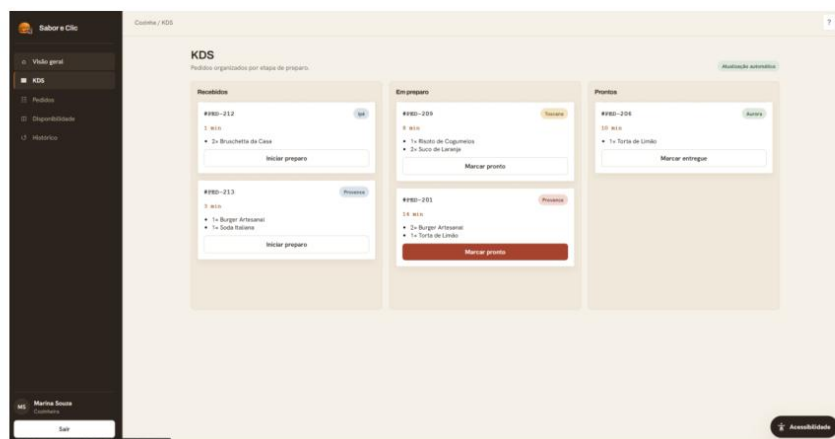
17.2 Telas do perfil Cozinheiro

Figura 11 – Protótipo: Visão geral da cozinha (Cozinheiro)



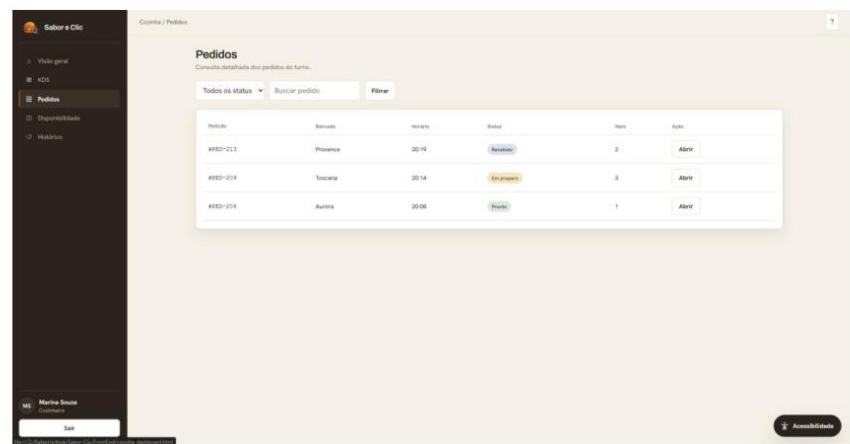
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 12 – Protótipo: KDS (Cozinheiro)



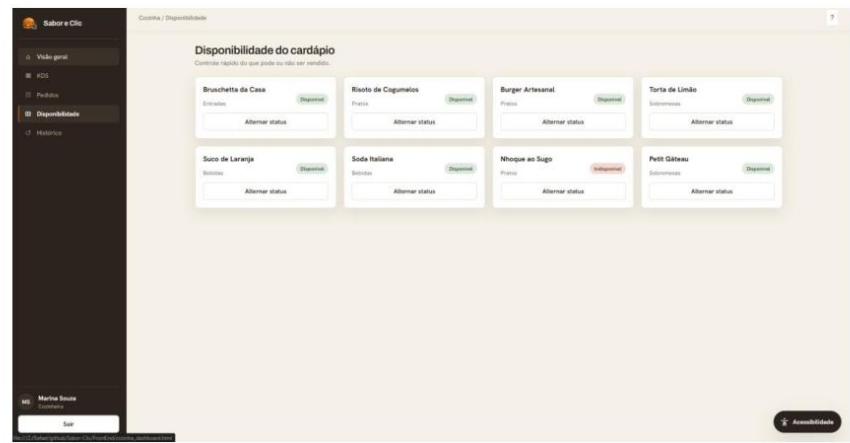
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 13 – Protótipo: Pedidos (Cozinheiro)



Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 14 – Protótipo: Disponibilidade do cardápio (Cozinheiro)



Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 15 – Protótipo: Histórico (Cozinheiro)

Pedido	Banquete	Início	Fim	Tempo
#PED-215	Toscana	19:42	19:53	11 min
#PED-149	Ipê	19:31	19:44	12 min
#PED-154	Provence	19:20	19:30	10 min

Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

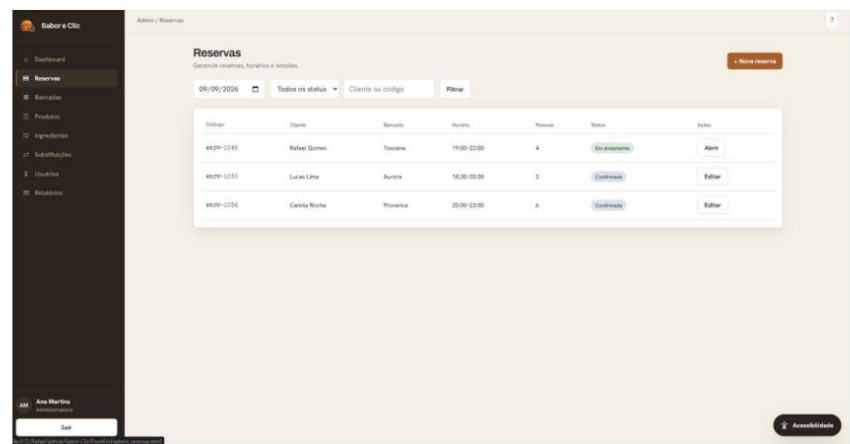
17.3 Telas do perfil Administrador

Figura 16 – Protótipo: Dashboard (Administrador)

Dashboard				
Visão geral do funcionamento do Sabor e Clic				
Reservas hoje 12 (2 no cancel)	Reservas ocupadas 3/6 80%	Pedidos ativos 7	Receita do dia R\$ 1.640,00 até	Clientes hoje 31
Operação atual				
Próximas reservas				
18:30 • Aurora 2 pessoas • Lucão Lima Confirmado				
19:00 • Toscana 4 pessoas • Rafael Gomes Em andamento				
20:00 • Provence 6 pessoas • Carlos Rocha Confirmado				
Alertas				
Reserva Bancada Cozinha em manutenção.				
Produto Whiskey ao Sugo indisponível.				
Estoque Cogumelos abaixo do estoque mínimo.				

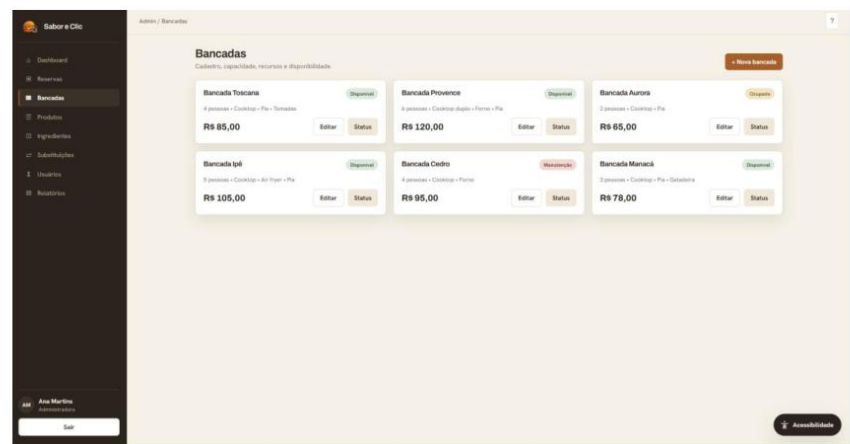
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 17 – Protótipo: Reservas (Administrador)



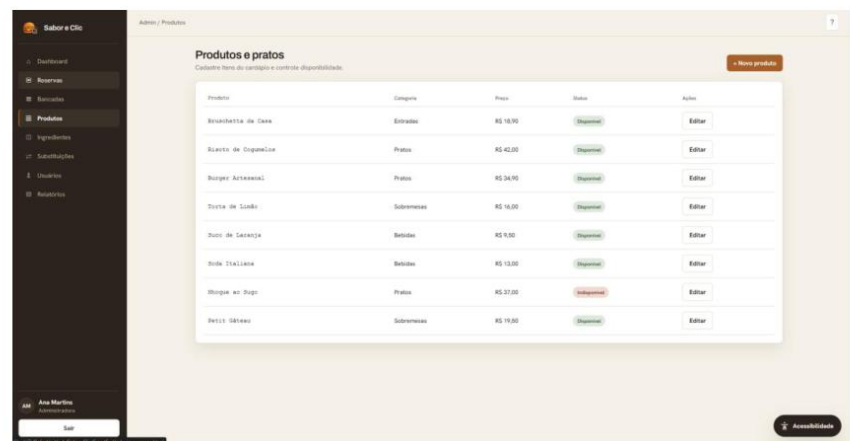
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 18 – Protótipo: Bancadas (Administrador)



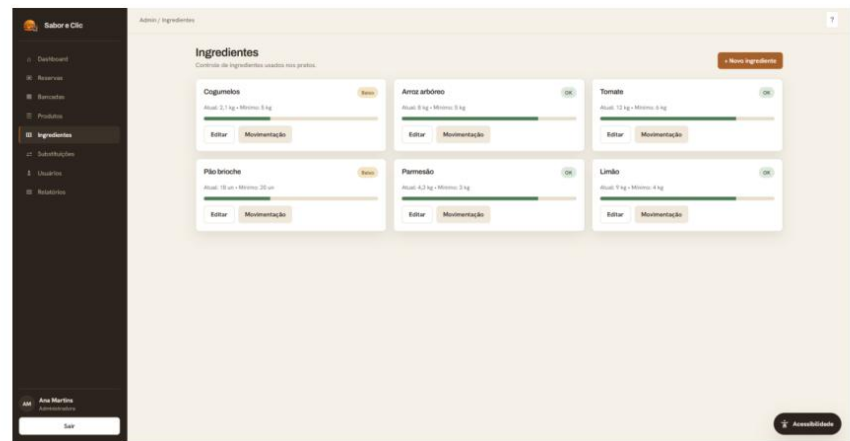
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 19 – Protótipo: Produtos e pratos (Administrador)



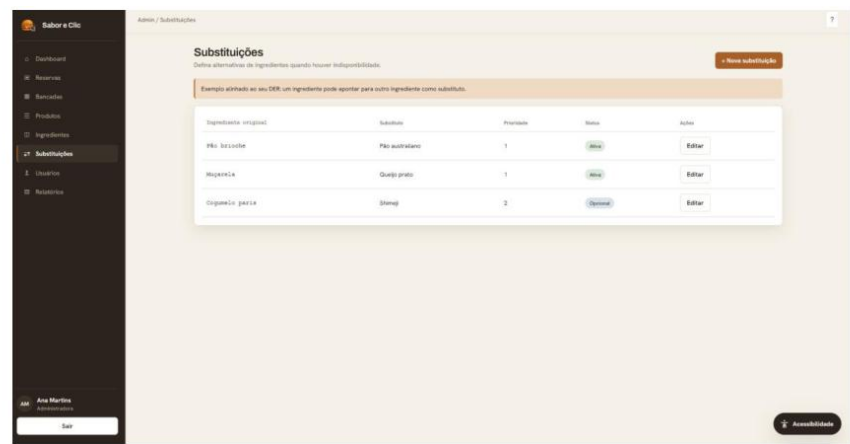
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 20 – Protótipo: Ingredientes (Administrador)



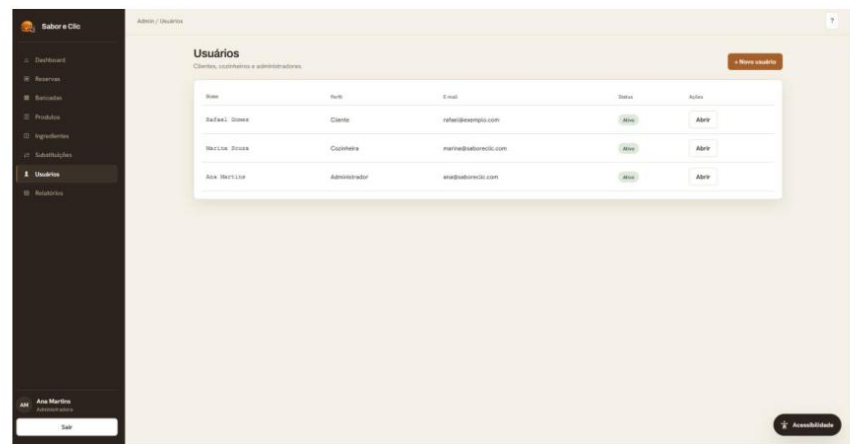
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 21 – Protótipo: Substituições (Administrador)



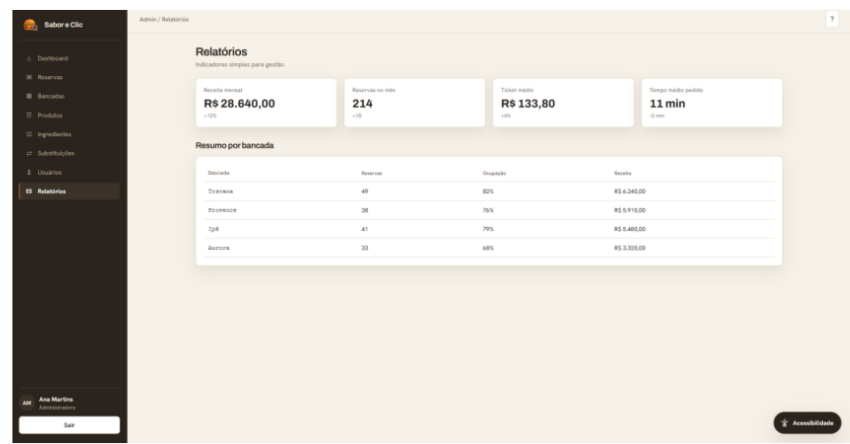
Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 22 – Protótipo: Usuários (Administrador)



Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

Figura 23 – Protótipo: Relatórios (Administrador)



Fonte: protótipo do Sabor e Clic elaborado pela equipe no Figma (2026).

18 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

Quadro 8 – Requisitos funcionais

Código	Requisito funcional	Estágio
RF01	Entrar e acessar a interface correspondente ao papel.	Demonstrado no protótipo; autenticação real é próxima etapa.
RF02	Consultar e reservar Bancadas.	Interface concluída; API cria e consulta Reservas em memória.
RF03	Criar, ativar e encerrar Sessão.	Interface e endpoints de criação, consulta e encerramento concluídos em memória.
RF04	Consultar Cardápio e manter carrinho.	Implementado em memória.
RF05	Criar e acompanhar Pedido.	Interface e API em memória concluídas; integração Fetch planejada.
RF06	Operar fila no KDS.	Layout e consulta JSON concluídos; transições persistentes planejadas.
RF07	Administrar catálogo, Ingredientes e usuários.	Telas demonstrativas concluídas; CRUD planejado.

Fonte: elaboração da equipe (2026).

Quadro 9 – Requisitos não funcionais

Requisito não funcional	Como tratamos
Usabilidade	Hierarquia, feedback, navegação por perfil e consistência visual.
Responsividade	Grid, Flexbox e media queries no CSS.
Manutenibilidade	Separação real entre Routes, Controllers e Models, além dos módulos JavaScript por responsabilidade.
Segurança	Perfis e validações de entrada demonstrados; autenticação, cookies e RBAC previstos.
Integridade	Validações em memória nesta etapa; PK, FK, unicidade e transações previstas no banco.
Desempenho	Cálculos e consultas em memória; índices e filtros no servidor ficam para a persistência.

Fonte: elaboração da equipe (2026).

19 TECNOLOGIAS UTILIZADAS

Quadro 10 – Tecnologias utilizadas e aplicações no projeto

Tecnologia	Aplicação no projeto
HTML5	Estrutura das páginas e formulários.
CSS3	Identidade visual, componentes e responsividade.
JavaScript ES6+	DOM, estado, arrays, templates, carrinho e cálculos.
JSON e Fetch API	JSON implementado na API Flask; Fetch permanece planejado para o quarto bimestre.
Python e Flask	API executável, validações e dados demonstrativos em memória.

Banco relacional/MySQL Workbench	DER, MER, EER e futura persistência SQL.
Git e GitHub	Versionamento e integração do trabalho da equipe.

Fonte: elaboração da equipe (2026).

20 O QUE CONCLUÍMOS E PRÓXIMAS ETAPAS

20.1 O que concluímos

Concluímos o protótipo navegável, a identidade visual, as telas dos três atores, o Cardápio, o carrinho em memória, a visualização de Pedidos, o layout do KDS, o DER, o MER e o dicionário de dados. Também modularizamos o JavaScript para evidenciar os conteúdos da etapa e implementamos uma API Flask executável com Blueprints, Controllers, Models em memória e respostas JSON.

20.2 O que iniciamos

Implementamos a factory Flask e endpoints reais para cadastro/login demonstrativos, Bancadas, Reservas, Sessões, Cardápio, Pedidos e KDS. O Pedido usa id_sessão; o servidor valida a Sessão ativa, recalcula valores e mantém o fluxo Reserva → Sessão → Pedido → KDS. Os dados são reiniciados com o servidor. Autenticação, banco físico e Fetch permanecem para a próxima etapa.

20.3 Próximas etapas

Nas próximas etapas, substituiremos os Models em memória por persistência relacional, implementaremos autenticação e RBAC, conectaremos o front-end à API por Fetch/JSON e persistiremos as transições do KDS. Também serão acrescentados POO completa em JavaScript, tratamento de loading/erros, constraints, consultas, triggers e testes de integração.

21 ATENDIMENTO AOS REQUISITOS DO TERCEIRO BIMESTRE

Quadro 11 – Atendimento aos requisitos do terceiro bimestre

Requisito	Situação	Evidência produzida
DER	Concluído	DER_saborclic.png, MER_saborclic.png e EER_saborclic.mwb.

Dicionário de dados	Concluído	Documento em DOCX e PDF com entidades, atributos e relações.
Wireframes/Figma	Concluído	Link público do Figma e protótipo navegável registrados.
Identidade visual	Concluída	Paleta, tipografia, marca e componentes em CSS.
Cardápio Digital	Concluído como protótipo	Produtos, categorias, imagens, disponibilidade e carrinho.
KDS	Concluído como protótipo	Colunas, tickets, tempos e ações simuladas.
JavaScript/DOM	Concluído para a etapa	validation.js e login.js validam no navegador; Controllers validam Reservas, Sessões e Pedidos no Flask.
Validações	Concluído para a etapa	Factory Flask, Blueprints, Routes, Controllers, Models em memória, JSON e fluxo Reserva → Sessão → Pedido → KDS.
Flask/MVC/rotas/JSON	Definido e demonstrado	Perfis e separação visual existem; autenticação real, cookies e RBAC serão aplicados no quarto bimestre.
Segurança e RBAC	Definido e demonstrado no protótipo	Perfis e separação visual existem; autenticação e RBAC de autoridade serão aplicados no servidor.

Fonte: elaboração da equipe (2026).

22 ESTADO ATUAL DA IMPLEMENTAÇÃO E INTEGRAÇÃO

Esta seção consolida a implementação atual. No terceiro bimestre, o front-end utiliza dados simulados para demonstrar as telas, enquanto o Flask expõe uma API JSON executável com dados separados em memória. No quarto bimestre, Fetch conectará essas partes e o banco substituirá a memória como fonte dos dados.

22.1 Front-end e JavaScript

O front-end possui áreas separadas para Cliente, Cozinheiro e Administrador. O JavaScript está dividido entre data.js, site.js, components.js, catalog.js, cart.js, validation.js, login.js, views.js e a11y.js, cobrindo arrays, objetos, for...of, for...in, forEach, DOM, validação e cálculos em memória.

22.2 Back-end Flask em memória

A factory Flask registra Blueprints para cadastro/login demonstrativos, Bancadas/Reservas, Sessões, Cardápio/Pedidos e KDS. As Routes recebem JSON, os Controllers validam os casos de uso e os Models mantêm dados simulados. Pedido referencia id_sessao, preservando a cadeia até a Reserva e a Bancada. Não há persistência ou autenticação real.

22.3 Integração e próxima etapa

O fluxo permanece Reserva → Sessão → Cardápio/Carrinho → Pedido → KDS. A API já recebe identificadores e quantidades, válida a Sessão, recalcula valores e devolve JSON quando chamada diretamente. Na integração do quarto bimestre, o JavaScript fará essas chamadas por Fetch, o banco persistirá os dados e o KDS enviará alterações de estado ao back-end.

REFERÊNCIAS

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO. **Projeto Integrador Sabor & Clic**: sistema de gestão para dark kitchens e espaços gastronômicos. Versão de 31 de agosto de 2026. São Paulo: IFSP, 2026.

SABOR E CLIC. **Dicionário de Dados Sabor e Clic**. São Paulo, 2026.

SABOR E CLIC. **Repositório Sabor-Clic**. GitHub, 2026. Disponível em: <https://github.com/FaellEmiliano/Sabor-Clic>.